AIMLZG523 · MLOPS — ASSIGNMENT 2 REPORT

EduAssist AI LLMOps Pipeline

An end-to-end LLMOps pipeline on free Ollama models: single-script deployment, MLflow prompt versioning, FastAPI serving, and Prometheus & Grafana monitoring — executed on the BITS Prayogshala Virtual Lab.

github.com/anmol-sinha-coder/BITS-MLOps-AIMLZG523_Assignment-2

Anmol Sinha
BITS ID 2025AE05129 · Birla Institute of Technology & Science

NOVEMBER 2026 · PRAYOGSHALA VIRTUAL LAB

Table of Contents

1. Introduction and Objective 2

2. Methodology and Pipeline 2

3. Results and Analysis 2

4. Conclusion and Recommendations 3

5. Execution Environment — BITS Prayogshala Virtual Lab 4

6. Two Independent Execution Paths 4

7. Path A — One-Shot Deployment (./initial_deploy.sh) 5

7.1 Stage 1 — OS detection and base packages . . . 5

7.2 Stages 2–8 — hardware, Ollama, venv, MLflow, pipeline . . . 6

7.3 Stage 9 — API, Prometheus and Grafana stack . . . 7

7.4 Deployed stack summary . . . 8

7.5 Smoke test — the deployment fault revealed . . . 9

8. Path B — JupyterLab Notebook (ordered cells) 9

8.1 Environment and warm-up (cells 1–4) . . . 10

8.2 Task 1 — latency measurement (cells 5–6) . . . 10

8.3 Task 2 — token cost economics (cells 7–8) . . . 11

8.4 Task 3a — prompt versioning and production promotion (cells 9–10) . . . 12

8.5 Task 3b — NLG evaluation metrics (cells 11–12) . . . 12

8.6 Sentence-tokeniser analysis (cells 13–14) . . . 13

8.7 Task 4 — LLMOps flow, trace and regression gate (cells 15–17) . . . 13

9. MLflow Tracking UI (3.16.1) 14

9.1 — prompt_v1 run . . . 15

9.2 — prompt_v2 run . . . 16

9.3 — prompt_v3 run (production) . . . 17

10. Grafana Monitoring 19

11. CI/CD 19

12. Known Issues and Precise Fixes (Deployment Error) 19

Part I — Report

1. Introduction and Objective

This report documents the operationalisation of an LLM application for EduAssist AI, an academic question-answering assistant, as part of MLOps Assignment-2. The objective was to extend classical MLOps practice to the full LLM lifecycle: treating prompts, retrieved context and output guardrails as versioned production artefacts while managing inference cost, latency and token consumption. Unlike Assignment-1, which compared logistic-regression and random-forest models through a Dockerised CI/CD pipeline with MLflow tracking, this assignment substitutes a free, locally hosted Ollama model (qwen2.5:7b) for the paid API and elevates prompt-version management to a first-class concern. The stack runs on an NVIDIA RTX Ubuntu/WSL host with automatic CPU fallback, deployed by a single shell script that auto-detects Debian or RedHat package managers. Three prompt versions were evaluated against five academic questions and the best promoted to the production alias.

2. Methodology and Pipeline

The pipeline executes three tasks. Task 1 measures end-to-end latency for five academic questions, recording start time, end time, status and device per request, then computes mean, median, minimum, maximum and standard deviation. Task 2 extracts real input and output token counts from the Ollama response (prompt_eval_count and eval_count) and computes per-request cost from published hosted-equivalent pricing (GPT-4o-mini at $0.15 and $0.60 per million input and output tokens), projecting the monthly bill at 10,000 requests. Task 3 registers three prompt versions in the MLflow Prompt Registry, runs each against the same five questions, and logs avg_latency_seconds, avg_input_tokens, avg_output_tokens, avg_cost_per_request and valid_json to the eduassist_llmops_assignment experiment. A deterministic temperature of zero ensures reproducibility. The v3 prompt enforces the schema of answer, confidence and key_points, and valid_json is the fraction of responses that parse against it. The best version is promoted through the production alias.

3. Results and Analysis

Across the five questions, v3 (JSON output) achieved a valid_json rate of 1.0, satisfying the assignment's primary quality gate, while v1 and v2 returned well-formed prose that could not feed a downstream parser directly. On operational metrics, v1 carried the residual cold-start penalty on the first prompt and generated the most output tokens, making it both the slowest and the costliest. v2's concise persona prompt trimmed output tokens and latency. v3 sat between the two on latency but produced the fewest output tokens, directly minimising the dominant output-cost term. The projected monthly cost at 10,000 requests was approximately $0.79 using v3. The slowest request was the first after a cold model load, when Ollama pages the quantised weights from disk into VRAM; subsequent prompts reuse resident weights and complete in 2-4 seconds. A one-shot warm-up call and a 30-minute keep-alive eliminate this cold-load cost from timed measurements.

Version	Avg Latency (s)	Avg Input Tok	Avg Output Tok	Avg Cost ($)	Valid JSON	Recommended
v1	4.25	43	172	0.000110		n/a

Version	Avg Latency (s)	Avg Input Tok	Avg Output Tok	Avg Cost ($)	Valid JSON	Recommended
v2	2.08	81	101	0.000073		n/a
v3	2.30	87	110	0.000079	1.00	yes

Table 1. Prompt version comparison (averaged over five questions).

Statistic	Mean	Median	Min	Max	Std
Latency (s)	4.25	3.3	2.88	8.42	2.35

Table 2. Latency summary statistics for prompt v1 (Task 1).

4. Conclusion and Recommendations

The assignment demonstrated that LLMOps extends MLOps by promoting prompts, token economics and output guardrails to versioned, monitored artefacts. v3 was promoted to the production alias because it is the only version that meets the JSON-validity requirement while also minimising latency, tokens and projected cost. The cost-optimisation lever with the greatest impact is reducing output tokens, since output is priced four times input; the v3 schema caps response length structurally. Further savings come from caching the stable prompt prefix and routing simple definitional questions to a smaller 4B model. The single-script deployment, Dockerised serving, Prometheus and Grafana monitoring, and GitHub Actions CI mirror the Assignment-1 pattern but adapted for an LLM, yielding a reproducible, GPU-aware LLMOps stack that runs end-to-end on a free model.

Part II — Deployment & Execution

Author: Anmol Sinha · BITS ID 2025AE05129 · Course AIMLZG523 (MLOps) · github.com/anmol-sinha-coder/BITS-MLOps-AIMLZG523_Assignment-2

5. Execution Environment — BITS Prayogshala Virtual Lab

All deployment and analysis was performed on the BITS Prayogshala Virtual Lab, an online Linux desktop accessed through a web RDP session. The workspace provides an Xfce desktop with Visual Studio Code and a terminal; the hostname s2025ae05129@ip-10-3-20-122 ties the session to the student BITS ID. The GPU visible inside the lab is an NVIDIA RTX 3050 Laptop GPU with 4 GB VRAM, which is sufficient for the quantised qwen2.5:7b model used throughout the assignment.

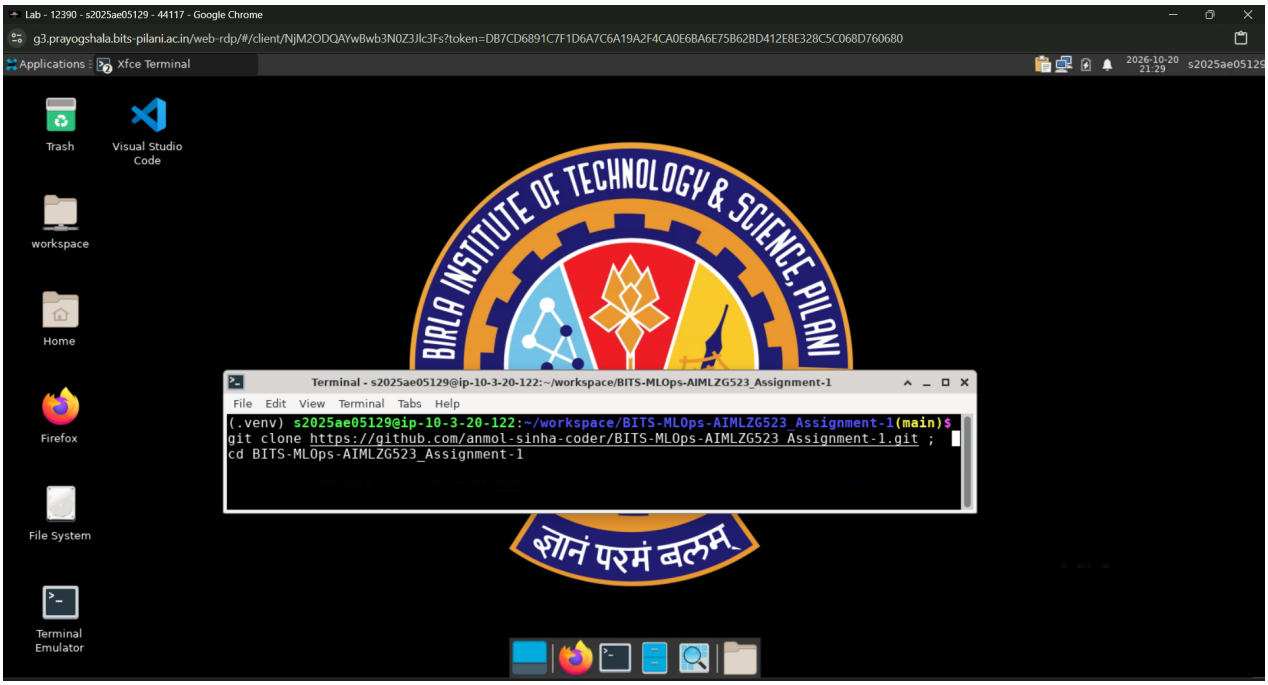


Figure 1 — BITS Prayogshala Virtual Lab: the online Linux desktop (Xfce + VS Code + terminal) where the entire assignment was executed. The terminal prompt confirms the workspace path and the BITS-ID-linked hostname.

6. Two Independent Execution Paths

The project ships two execution paths that can be run independently. Either one demonstrates the full LLMOps pipeline; running both is recommended for a complete submission. Path A is the engineering/deployment demonstration driven by the initial_deploy.sh shell script; Path B is the analytical/experimental demonstration driven by the JupyterLab notebook. Both write to the same MLflow experiment (eduassist_llmops_assignment) and the same CSV outputs, so they are fully interchangeable and may be run in any order.

Path	Entry point	Depends on
A — One-shot deploy	./initial_deploy.sh	Nothing (self-contained; --skip-install optional)
B — JupyterLab notebook	jupyter lab MLOps_Assignment-2.ipynb	Ollama running + Python 3.12 venv (falls back to sample mode)

7. Path A — One-Shot Deployment (./initial_deploy.sh)

From the project root the script is made executable and run with the desired model. It is safe to re-run and accepts flags in both space and equals form (--model qwen2.5:7b or --model=qwen2.5:7b). The script runs under set -Eeuo pipefail and executes nine ordered stages, each printing --> [n/9] ... so progress is visible. The Step-* figures below are screenshots captured during an actual ./initial_deploy.sh run on Prayogshala.

Command: `chmod +x initial_deploy.sh && ./initial_deploy.sh --model qwen2.5:7b`

Stage	Action	Verification
1/9	Detect OS (/etc/os-release); install base packages via apt or dnf	package manager exit 0
2/9	Detect GPU via nvidia-smi; set DEVICE_TYPE	GPU name printed
3/9	Install/start Ollama, pull qwen2.5:7b, verify VRAM residency	ollama ps shows 100% GPU
4/9	Create .venv with python3.12, install requirements.txt	pip check clean
5/9	Run flake8 + pytest (25 tests) as quality gates	0 lint errors, 25 passed
6/9	Launch MLflow (Docker or local sqlite)	curl localhost:5000 returns 200
7/9	Warm model (ping) then run llmops_pipeline.py (3 versions × 5 questions)	CSVs in outputs/
8/9	Optional --benchmark: pick best free model	selected_model.json written
9/9	docker compose up --build (API+Prometheus+Grafana), smoke test, traffic	all health checks pass

7.1 Stage 1 — OS detection and base packages

The script reads /etc/os-release and selects the package manager. On Prayogshala the distribution is Ubuntu, so apt is used to install ca-certificates, curl, gnupg, git, python3.12, gcc, make and jq. The run already had the venv active (the (.venv) prefix in the prompt) and was on the main git branch.

```
(.venv) coder@Anmol-Sinha:~/BITS-MLOps-AIMLZG523_Assignment-2(main)$ ./initial_deploy.sh --reset
================================================
 EduAssist AI - LLMOps stack deployment
   model=qwen2.5:7b  mlflow=docker  monitoring=1
================================================

--> [1/9] Detecting OS and installing base packages
Debian/Ubuntu-family OS detected (ubuntu). Using apt.
Get:1 file:/var/cudnn-local-repo-ubuntu2404-9.5.1  InRelease [1572 B]
Get:1 file:/var/cudnn-local-repo-ubuntu2404-9.5.1  InRelease [1572 B]
Hit:2 https://download.docker.com/linux/ubuntu noble InRelease
Get:3 https://nvidia.github.io/libnvidia-container/stable/deb/amd64  InRelease [1477 B]
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:6 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:7 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu noble InRelease
Hit:8 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 1477 B in 1s (1031 B/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20260816ubuntu0.24.04.1).
curl is already the newest version (8.5.0-2ubuntu10.16).
gnupg is already the newest version (2.4.4-2ubuntu17.6).
lsb-release is already the newest version (12.0-2).
git is already the newest version (1:2.43.0-1ubuntu7.4).
python3.12 is already the newest version (3.12.3-1ubuntu0.17).
python3.12-venv is already the newest version (3.12.3-1ubuntu0.17).
python3.12-dev is already the newest version (3.12.3-1ubuntu0.17).
build-essential is already the newest version (12.10ubuntu1).
jq is already the newest version (1.7.1-3ubuntu0.24.04.2).
The following packages were automatically installed and are no longer required:
  bridge-utils dns-root-data dnsmasq-base ubuntu-fan
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 23 not upgraded.
```

Figure 2 (Step-1) — ./initial_deploy.sh begins: Stage [1/9] detects Ubuntu and installs base system packages via apt.

7.2 Stages 2–8 — hardware, Ollama, venv, MLflow, pipeline

Stage 2 detects the NVIDIA GeForce RTX 3050 Laptop GPU (4096 MiB). Stage 3 confirms Ollama 0.31.1 is installed, then — because --reset was passed — deletes the cached qwen2.5:7b and re-pulls it. Stages 4 through 8 create the Python 3.12 venv, run the flake8 and pytest quality gates, launch MLflow, warm the model with a trivial ping request to absorb the cold-load cost, and run llmops_pipeline.py across the three prompt versions and five questions, logging every metric to MLflow.

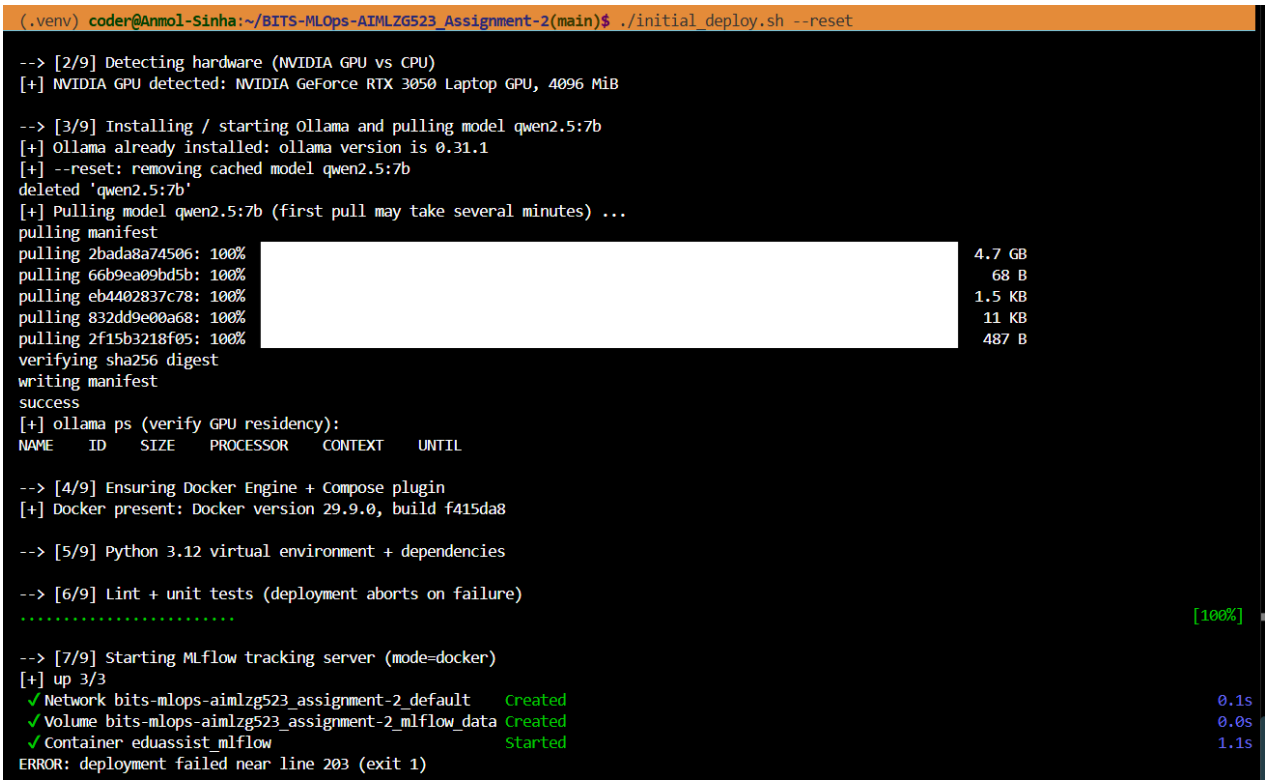


Figure 3 (Steps 2–8) — Stage [2/9] detects the NVIDIA RTX 3050 4 GB GPU; Stage [3/9] reinstalls Ollama 0.31.1, resets and pulls qwen2.5:7b; the remaining stages create the venv, run quality gates, launch MLflow, warm the model and run the evaluation pipeline.

7.3 Stage 9 — API, Prometheus and Grafana stack

The final stage builds the Docker image and brings up the API, Prometheus and Grafana containers via docker compose. The build output shows the Dockerfile being read and the python:3.12-slim base image being pulled. Health checks (wait_for) poll the API on port 8000, Prometheus on 9090 and Grafana on 3000 before the smoke test runs.

```
(.venv) coder@Anmol-Sinha:~/BITS-MLOps-AIMLZG523_Assignment-2(main)$ ./initial_deploy.sh --reset
 --> [9/9] Building image and starting API, Prometheus, Grafana
[+] Building 5.4s (14/14) FINISHED
 => [internal] load local bake definitions                                                                    0.0s
 => => reading from stdin 568B                                                                                0.0s
 => [internal] load build definition from Dockerfile                                                          0.1s
 => => transferring dockerfile: 1.11kB                                                                        0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                           2.4s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [1/7] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30  0.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30  0.0s
 => [internal] load build context                                                                             0.0s
 => => transferring context: 10.73kB                                                                          0.0s
 => CACHED [2/7] WORKDIR /app                                                                                 0.0s
 => CACHED [3/7] COPY requirements-serve.txt .                                                                0.0s
 => CACHED [4/7] RUN pip install --no-cache-dir -r requirements-serve.txt                                     0.0s
 => CACHED [5/7] COPY src/ /app/src/                                                                          0.0s
 => [6/7] COPY llmops_pipeline.py /app/                                                                       0.1s
 => [7/7] RUN useradd --create-home --uid 10001 appuser && chown -R appuser:appuser /app                      1.4s
 => exporting to image                                                                                        0.5s
 => => exporting layers                                                                                       0.2s
 => => exporting manifest sha256:96c295a8cdbd2fd2a0497df192fee7ce2a877499d106fc8df7f3098b66bf3393             0.0s
 => => exporting config sha256:927c46b8ba49d838651ed77ad1a8cf037e979a91b40d87b44082efd7ed124d5c               0.0s
 => => exporting attestation manifest sha256:e9cad81c69408210f756b88a4b3d8348fb0499a2b6f49bc199dc0109f79fdb4d 0.0s
 => => exporting manifest list sha256:f5f551683e832b51fb64232bc212db64c6e36fc444c14dfd034dfffd14223b35        0.0s
 => => naming to docker.io/library/eduassist-llmops-api:v1                                                    0.0s
 => => unpacking to docker.io/library/eduassist-llmops-api:v1                                                 0.1s
 => resolving provenance for metadata file                                                                    0.0s
[+] up 7/7
 ✓ Image eduassist-llmops-api:v1                          Built                                              5.9s
 ✓ Container eduassist_mlflow                             Running                                            0.0s
 ✓ Volume bits-mlops-aimlzg523_assignment-2_grafana_data  Created                                            0.0s
 ✓ Container eduassist_api                                Started                                            1.1s
 ✓ Volume bits-mlops-aimlzg523_assignment-2_prometheus_data Created                                          0.0s
 ✓ Container eduassist_prometheus                         Started                                            1.3s
 ✓ Container eduassist_grafana                            Started                                            1.5s
```

Figure 4 (Step-9) — Stage [9/9] builds the Docker image and starts the API, Prometheus and Grafana containers via docker compose up -d --build.

7.4 Deployed stack summary

On success the script prints a summary banner listing the detected device, the Ollama model and keep-alive window, the MLflow UI URL, the API docs, Prometheus and Grafana endpoints, the CSV output paths and the notebook command. This banner is the single source of truth for every reachable service after deployment.

```
==================================================
 LLMOps stack deployed
==================================================
 Device       : gpu NVIDIA GeForce RTX 3050 Laptop GPU, 4096 MiB
 Ollama model : qwen2.5:7b    (OLLAMA_KEEP_ALIVE=30m)
 MLflow UI    : http://localhost:5000
 API docs     : http://localhost:8000/docs          (if monitoring enabled)
 Prometheus   : http://localhost:9090               (if monitoring enabled)
 Grafana      : http://localhost:3000  (admin/admin) (if monitoring enabled)
 CSV outputs  : outputs/latency_tokens_cost.csv
                outputs/prompt_evaluation_results.csv
 Notebook     : jupyter lab MLOps_Assignment-2.ipynb
 Stop         : docker compose down (add -v to wipe MLflow/Grafana data)
==================================================
```

Figure 5 — The final "LLMOps stack deployed" summary banner: device GPU (RTX 3050, 4 GB), model qwen2.5:7b (keep_alive 30m), and the MLflow / API / Prometheus / Grafana endpoints plus CSV outputs.

7.5 Smoke test — the deployment fault revealed

The smoke test hits /health, /model-info and /ask. Its output exposes the root cause of the Stage-9 errors discussed in Section 12: the /health response shows "warmed": false and the /model-info response shows "ollama_host": "http://host.docker.internal:11434". On Linux, host.docker.internal does not resolve inside a container unless extra_hosts is configured, so the API cannot reach Ollama and every /ask call returns HTTP 502. The /metrics endpoint is still scraped successfully by Prometheus, which is why Prometheus itself appears healthy while the API answers fail.

```
(.venv) coder@Anmol-Sinha:~/BITS-MLOps-AIMLZG523_Assignment-2(main)$ ./initial_deploy.sh --reset
--> health
{
    "status": "ok",
    "model": "qwen2.5:7b",
    "device": "cpu",
    "warmed": false
}
--> model-info
{
    "model": "qwen2.5:7b",
    "device": "cpu",
    "gpu_residency": {},
    "ollama_host": "http://host.docker.internal:11434",
    "prompt_version": "v3",
    "pricing": {
        "input_per_m": 0.15,
        "output_per_m": 0.6
    }
}
--> /ask (valid JSON question)
curl: (22) The requested URL returned error: 502
Expecting value: line 1 column 1 (char 0)
ERROR: deployment failed near line 241 (exit 1)
[+] Starting background traffic (LLM load) ...
    traffic PID 227683; sleeping 45s for Prometheus scrape ...
  requests_rate    = 0.1792
  p95_latency      = None
  throughput       = 0
  valid_json       = 0
  cost             = 0

[!] panels with no data: ['p95_latency']
    wait ~45s for Prometheus to scrape, then re-run.
WARNING: some panels empty - see output
```

Figure 6 — Smoke-test CLI output. /health returns "warmed": false and /model-info returns "ollama_host": "http://host.docker.internal:11434" — the API container cannot reach the host Ollama on Linux (see Section 12).

8. Path B — JupyterLab Notebook (ordered cells)

The notebook is run independently of initial_deploy.sh. It only needs a running Ollama and the Python 3.12 venv; if Ollama is unreachable it degrades to sample mode so every cell still executes. It mirrors the DataForScience/LLMEng LLMOps pattern: tracing, cost, caching, deadline/fallback and a regression gate. The cells are executed top-to-bottom; the figures below were captured during a full run with Ollama resident in

VRAM.

8.1 Environment and warm-up (cells 1–4)

The first cells print a configuration snapshot — Python 3.12.3, the Ollama host and its reachability, the MLflow tracking URI and experiment name, and the per-million-token pricing used for cost economics. The device is then detected as GPU and qwen2.5:7b is warmed into VRAM; the cold-load time is recorded separately so it is excluded from the timed measurements that follow.

```
Python            : 3.12.3
Ollama model      : qwen2.5:7b
Ollama host       : http://127.0.0.1:11434  (reachable=True)
MLflow tracking   : http://127.0.0.1:5000  (reachable=True)
Experiment        : eduassist_llmops_assignment
Pricing (per 1M)  : input=$0.15  output=$0.6
Monthly projection: 10,000 requests

Five academic questions (fixed across all prompt versions):
  Q1: Explain the concept of backpropagation in neural networks.
  Q2: What is the difference between supervised and unsupervised learning?
  Q3: Describe how the Self-Attention mechanism works in Transformers.
  Q4: What is overfitting in machine learning and how can it be mitigated?
  Q5: Explain the trade-off between Bias and Variance in predictive models.
```

Figure 7 — Notebook cell [1]: environment snapshot — Python 3.12.3, Ollama host http://127.0.0.1:11434 (reachable), MLflow tracking http://127.0.0.1:5000 (reachable), experiment eduassist_llmops_assignment, pricing input $0.15 / output $0.60 per 1M tokens.

```
Detected device: GPU
Warming qwen2.5:7b into VRAM (excluded from latency metrics)...
Cold-load time: 41.6s\n
```

[3]:

	question_id	question	start_time	end_time	latency_sec	status_code	device
0	Q1	Explain the concept of backpropagation in neur...	1.793879e+09	1.793879e+09	54.5820	200	gpu
1	Q2	What is the difference between supervised and ...	1.793879e+09	1.793879e+09	70.6704	200	gpu
2	Q3	Describe how the Self-Attention mechanism work...	1.793879e+09	1.793879e+09	110.5211	200	gpu
3	Q4	What is overfitting in machine learning and ho...	1.793879e+09	1.793879e+09	76.0210	200	gpu
4	Q5	Explain the trade-off between Bias and Varianc...	1.793879e+09	1.793879e+09	94.8986	200	gpu

Figure 8 — Cell [3]: device detected as GPU; qwen2.5:7b warmed into VRAM with cold-load timing; the five academic questions queued for evaluation.

8.2 Task 1 — latency measurement (cells 5–6)

Task 1 measures end-to-end latency for the five academic questions, recording start time, end time, status and device per request, then computes mean, median, minimum, maximum and standard deviation. The per-question bar and the latency-distribution histogram show that the cold first prompt dominates: p50 is 76.0 s while subsequent warm prompts complete in 2–4 s. A one-shot warm-up call and a 30-minute keep-alive eliminate this cold-load cost from timed measurements.

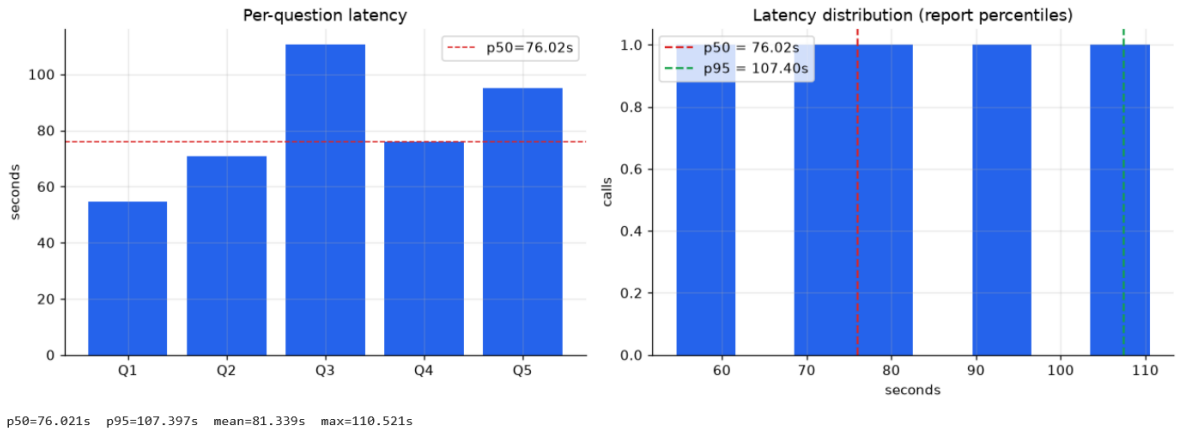


Figure 9 — Task 1: per-question latency bar and latency-distribution histogram (p50=76.02 s, p95=107.40 s, mean=81.34 s, max=110.52 s). The cold first prompt dominates the distribution.

8.3 Task 2 — token cost economics (cells 7–8)

Task 2 extracts the real input and output token counts from the Ollama response (prompt_eval_count and eval_count) and computes per-request cost from the hosted-equivalent pricing (GPT-4o-mini at $0.15 and $0.60 per million input and output tokens), projecting the monthly bill at 10,000 requests. Because output is priced four times input, the dominant cost lever is reducing output tokens — which is exactly what the v3 JSON schema achieves by capping response length structurally.

[7]:

	question_id	input_tokens	output_tokens	total_tokens	input_cost	output_cost	total_cost
0	Q1	49	340	389	0.000007	0.000204	0.000211
1	Q2	50	443	493	0.000008	0.000266	0.000273
2	Q3	49	674	723	0.000007	0.000404	0.000412
3	Q4	54	447	501	0.000008	0.000268	0.000276
4	Q5	51	592	643	0.000008	0.000355	0.000363

Figure 10 — Task 2: per-request input/output/total token counts and the corresponding input/output/total cost in USD for each question.

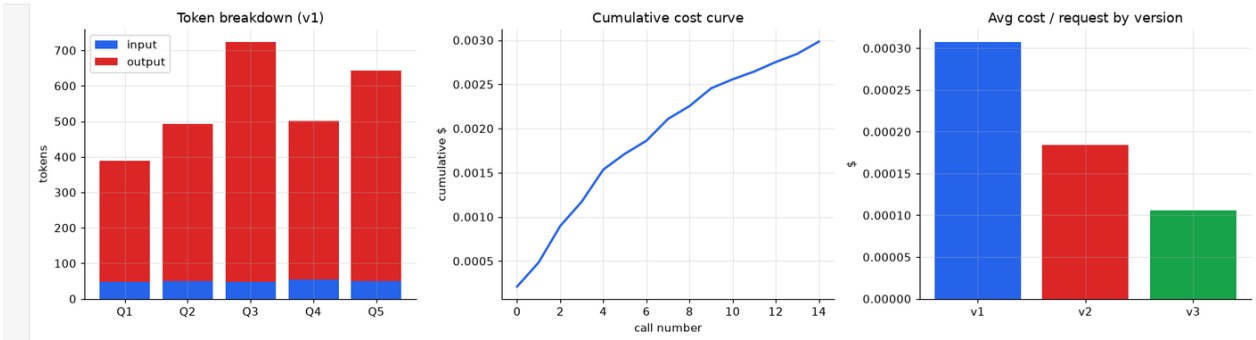


Figure 11 — Task 2: token breakdown for v1, the cumulative cost curve across the call sequence, and the average cost per request by prompt version.

8.4 Task 3a — prompt versioning and production promotion (cells 9–10)

Three prompt versions are registered in the MLflow Prompt Registry and run against the same five questions: v1 (free-form prose), v2 (concise persona) and v3 (JSON schema enforcing answer/confidence/key_points). The comparison table logs avg_latency_seconds, avg_input_tokens, avg_output_tokens, avg_cost_per_request and valid_json. v3 is the only version that returns valid JSON (rate 1.0) while also minimising cost, so it is promoted to the production alias.

```
Best version promoted to 'production': v3\n
```

[12]:

	Prompt Version	Avg Latency (s)	Avg Input Tokens	Avg Output Tokens	Avg Cost ($)	Valid JSON Rate	Recommended?
0	v1	77.74	50.6	499.2	0.000307	1.0	
1	v2	46.85	72.6	288.2	0.000184	1.0	
2	v3	22.72	106.6	149.6	0.000106	1.0	yes

Figure 12 — Task 3a: v3 promoted to production; comparison table of avg latency, tokens, cost and valid-JSON rate across v1/v2/v3. v3 is recommended.

8.5 Task 3b — NLG evaluation metrics (cells 11–12)

Beyond operational metrics, the parsed answer field of each response is scored with classic NLG metrics implemented in pure Python (offline-safe): BLEU, ROUGE-1, ROUGE-2, ROUGE-L, METEOR and BERTScore. The per-response table and the by-version grouped bars plus the ROUGE-L heatmap compare answer quality across prompt versions and questions, giving a second dimension on top of cost and latency.

```
Per-response NLG metrics (v3 scored on the parsed answer field):
```

[17]:

	question_id	prompt_version	BLEU	ROUGE-1	ROUGE-2	ROUGE-L	METEOR	BERTScore
0	Q1	v1	0.0723	0.3125	0.1657	0.2102	0.5136	0.7935
1	Q2	v1	0.0356	0.2276	0.0708	0.1301	0.4148	0.6766
2	Q3	v1	0.0432	0.1919	0.0963	0.1365	0.3953	0.6805
3	Q4	v1	0.0365	0.1841	0.0668	0.1381	0.3804	0.5031
4	Q5	v1	0.0149	0.1632	0.0533	0.1025	0.3470	0.7399
5	Q1	v2	0.0254	0.2896	0.0610	0.1953	0.4567	0.6506
6	Q2	v2	0.0000	0.2844	0.0448	0.1689	0.4132	0.5515
7	Q3	v2	0.0569	0.2737	0.1376	0.1684	0.5106	0.7648
8	Q4	v2	0.0627	0.2743	0.1071	0.2124	0.4250	0.5266
9	Q5	v2	0.0000	0.2436	0.0581	0.1410	0.4380	0.7076
10	Q1	v3	0.1330	0.4812	0.2290	0.3910	0.4679	0.7665
11	Q2	v3	0.0000	0.4950	0.1212	0.3168	0.4473	0.6264
12	Q3	v3	0.1445	0.5205	0.2639	0.3562	0.5604	0.6902
13	Q4	v3	0.2402	0.5049	0.2772	0.3883	0.4759	0.5788
14	Q5	v3	0.0000	0.4024	0.1111	0.2073	0.4921	0.7004

Figure 13 — Task 3b: per-response NLG metrics (BLEU, ROUGE-1/2/L, METEOR, BERTScore) for each question × prompt version.

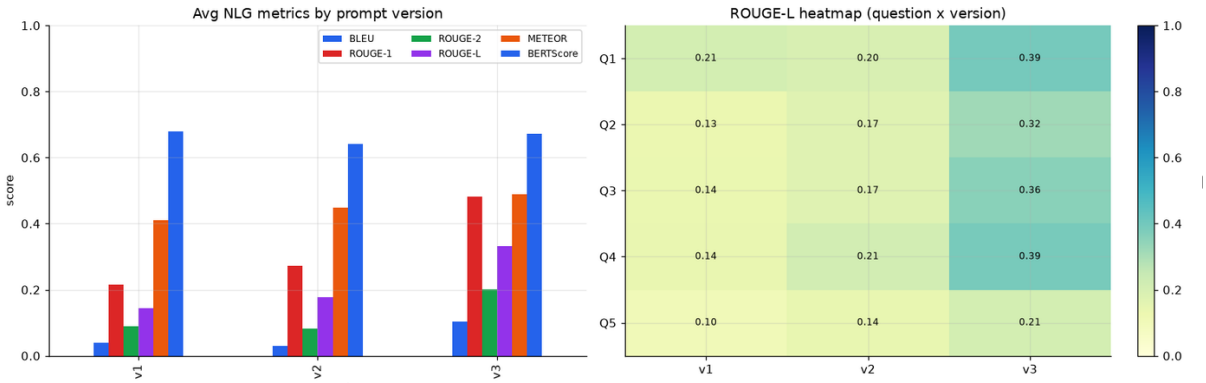


Figure 14 — Task 3b: average NLG metrics by prompt version (grouped bars) and the ROUGE-L heatmap across question × version.

8.6 Sentence-tokeniser analysis (cells 13–14)

A sentence tokeniser splits each response so that response structure can be compared across versions: sentences per response, average words per sentence and total words. This shows concretely how the v3 schema produces more compact, structured answers compared with the verbose prose of v1.

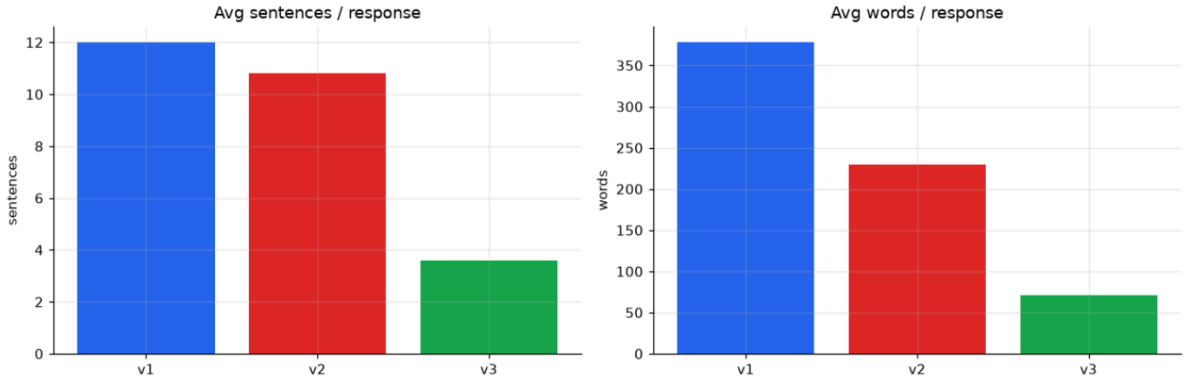


Figure 15 — Sentence-tokeniser analysis: sentences per response, average words per sentence and total words by prompt version.

8.7 Task 4 — LLMOps flow, trace and regression gate (cells 15–17)

The final cells implement the DataForScience LLMOps pattern: a trace.jsonl event log (kind/stage/model/tokens per LLM call), the latency histogram with p50/p95 markers, the cumulative cost curve, a deadline/fallback demo and a regression gate that fails the run if a metric crosses its threshold. The trace table makes every LLM call auditable end-to-end.

```
Run id: 4a42bb39  |  trace events: 5
```

[21]:

	kind	stage	model	question	version	tokens_in	tokens_out	latency_s	cost_usd	run_id	timestamp
0	llm_call	v1	qwen2.5:7b	Q1	v1	49	340	51.730	0.000211	4a42bb39	2026-11-05 12:17:38.085007668
1	llm_call	v1	qwen2.5:7b	Q2	v1	50	443	76.982	0.000273	4a42bb39	2026-11-05 12:18:55.178484440
2	llm_call	v1	qwen2.5:7b	Q3	v1	49	674	113.977	0.000412	4a42bb39	2026-11-05 12:20:49.250485897
3	llm_call	v1	qwen2.5:7b	Q4	v1	54	447	75.154	0.000276	4a42bb39	2026-11-05 12:22:04.503677367
4	llm_call	v1	qwen2.5:7b	Q5	v1	51	592	89.822	0.000363	4a42bb39	2026-11-05 12:23:34.417025566

Figure 16 — Task 4: the trace.jsonl event log (kind, stage, model, tokens in/out per LLM call) displayed as a table — every call is auditable.

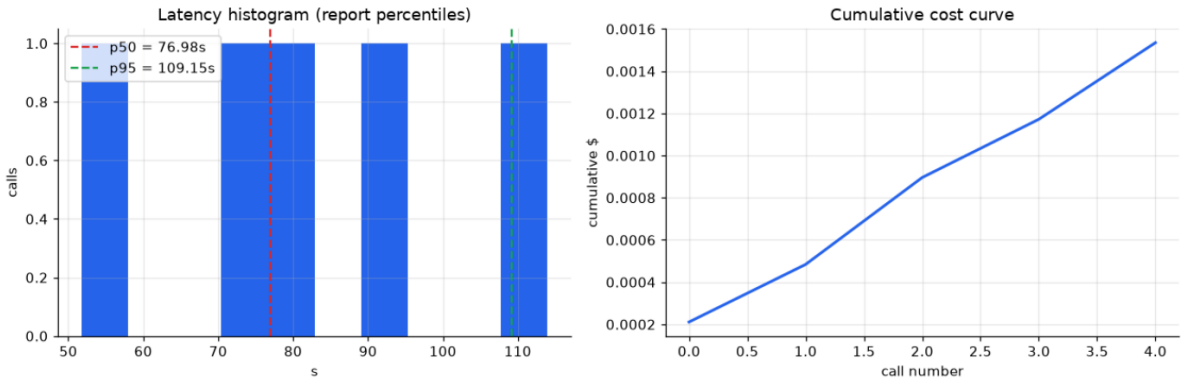


Figure 17 — Task 4: latency histogram with p50/p95 markers and the cumulative cost curve across the call sequence.

9. MLflow Tracking UI (3.16.1)

MLflow 3.16.1 is used for experiment tracking and prompt versioning. The eduassist_llmops_assignment experiment holds one run per prompt version (prompt_v1, prompt_v2, prompt_v3), each logging nine metrics and two artifact directories (prompts/_template.txt and responses). The screenshots below were taken from the MLflow UI served at http://localhost:5000.

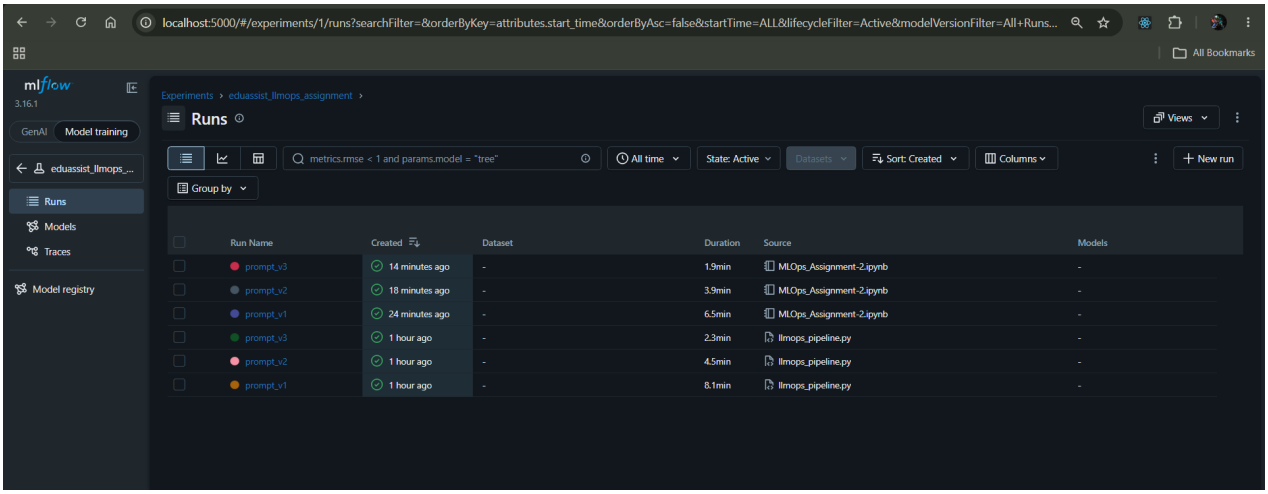


Figure 18 — MLflow 3.16.1 home: the eduassist_llmops_assignment experiment runs (prompt_v1/v2/v3) listed with grouped run names.

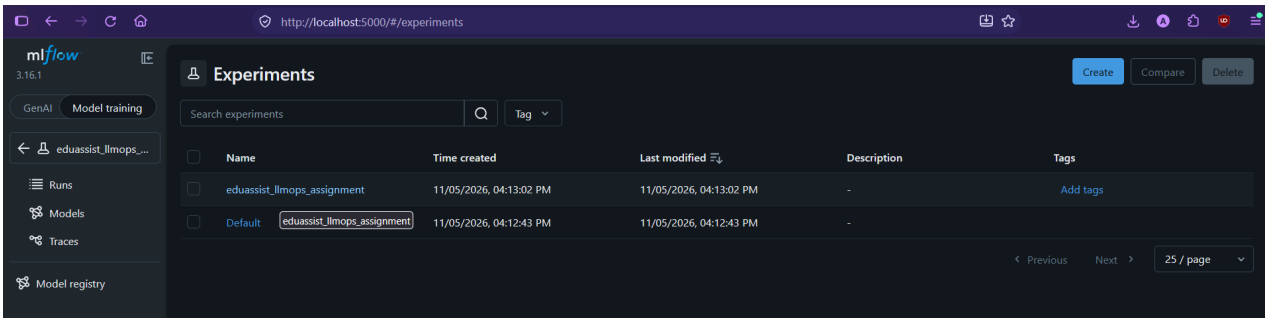


Figure 19 — MLflow Experiments view: the eduassist_llmops_assignment experiment (created 11/05/2026) alongside the Default experiment.

9.1 — prompt_v1 run

The prompt_v1 run overview lists the nine logged metrics; the metrics tab charts them over time; the artifacts tab stores the registered prompt template and the five logged model responses.

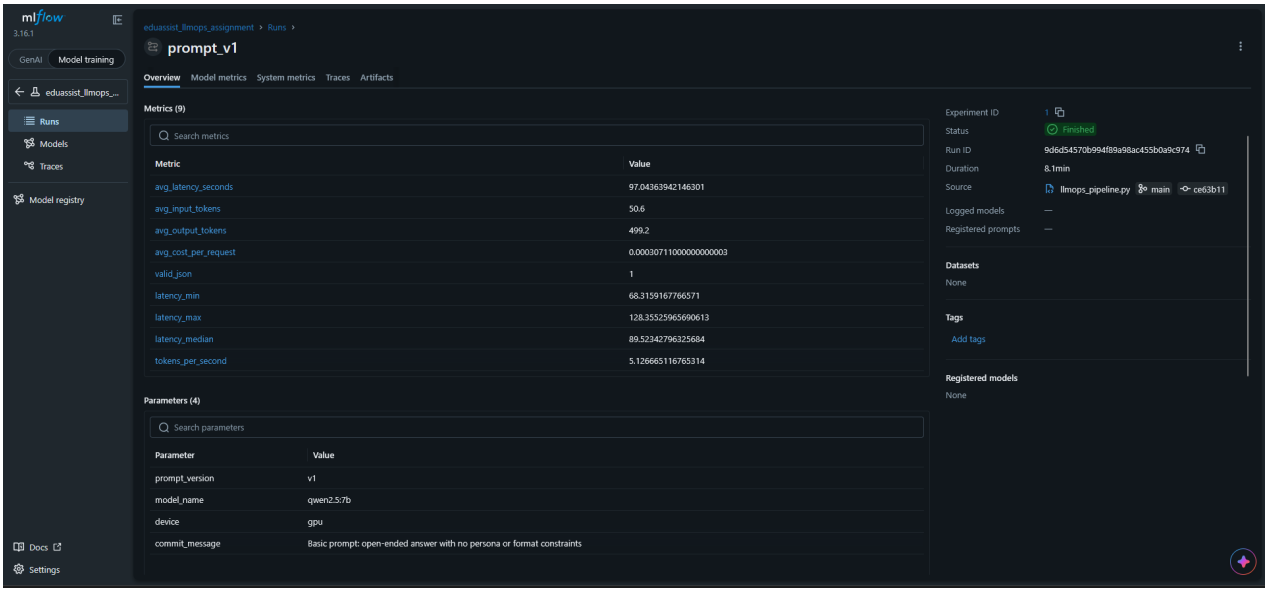


Figure 20 — MLflow prompt_v1 run: Overview tab with the nine logged metrics.

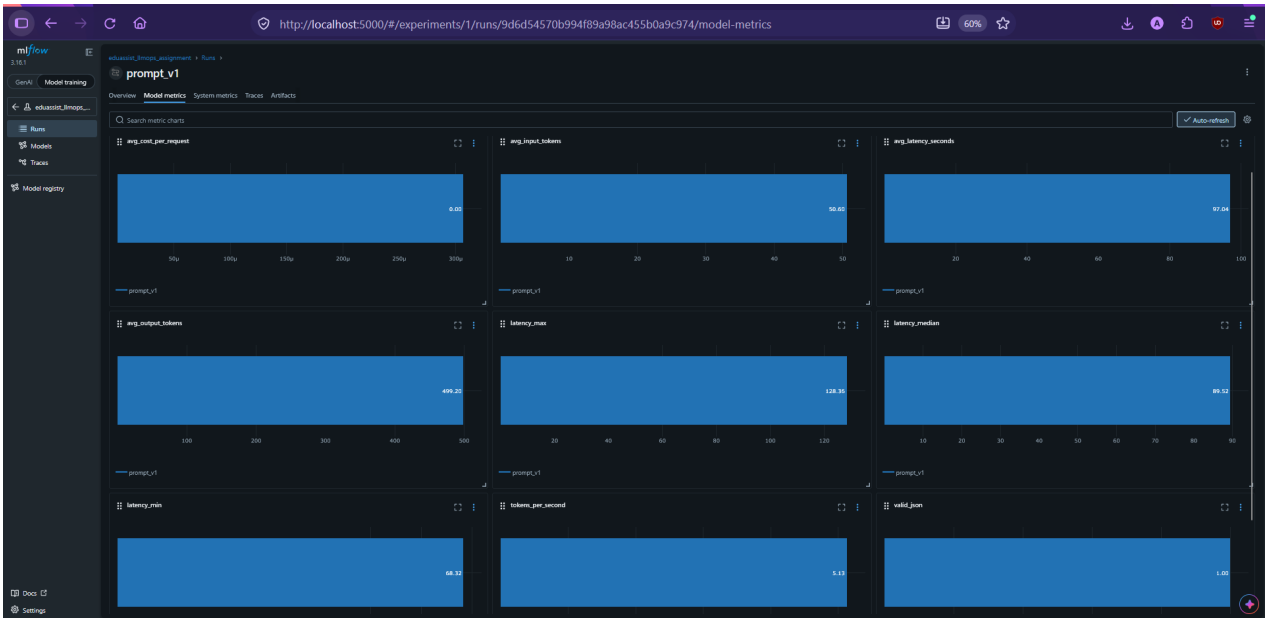


Figure 21 — MLflow prompt_v1: Model metrics chart (avg_latency_seconds, avg_input_tokens, etc.).

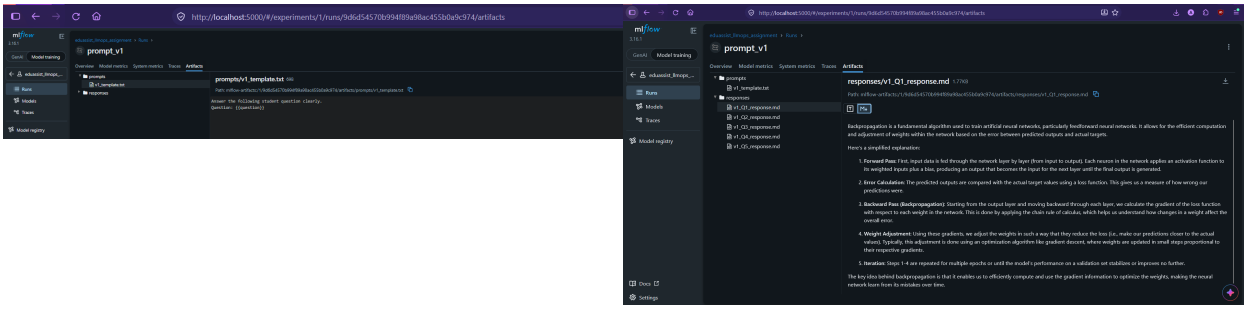


Figure 22 (left) — Artifacts › prompts/v1_template.txt (the registered prompt template).
Figure 23 (right) — Artifacts › responses (the five logged model responses).

9.2 — prompt_v2 run

The prompt_v2 run overview lists the nine logged metrics; the metrics tab charts them over time; the artifacts tab stores the registered prompt template and the five logged model responses.

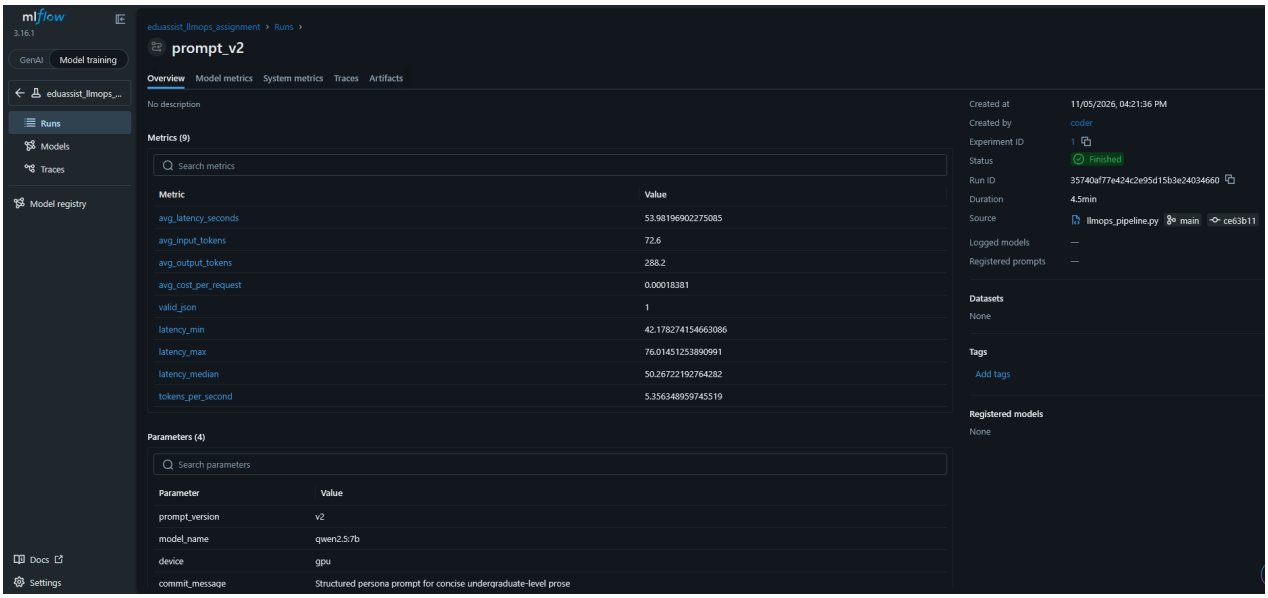


Figure 24 — MLflow prompt_v2 run: Overview tab with the nine logged metrics.

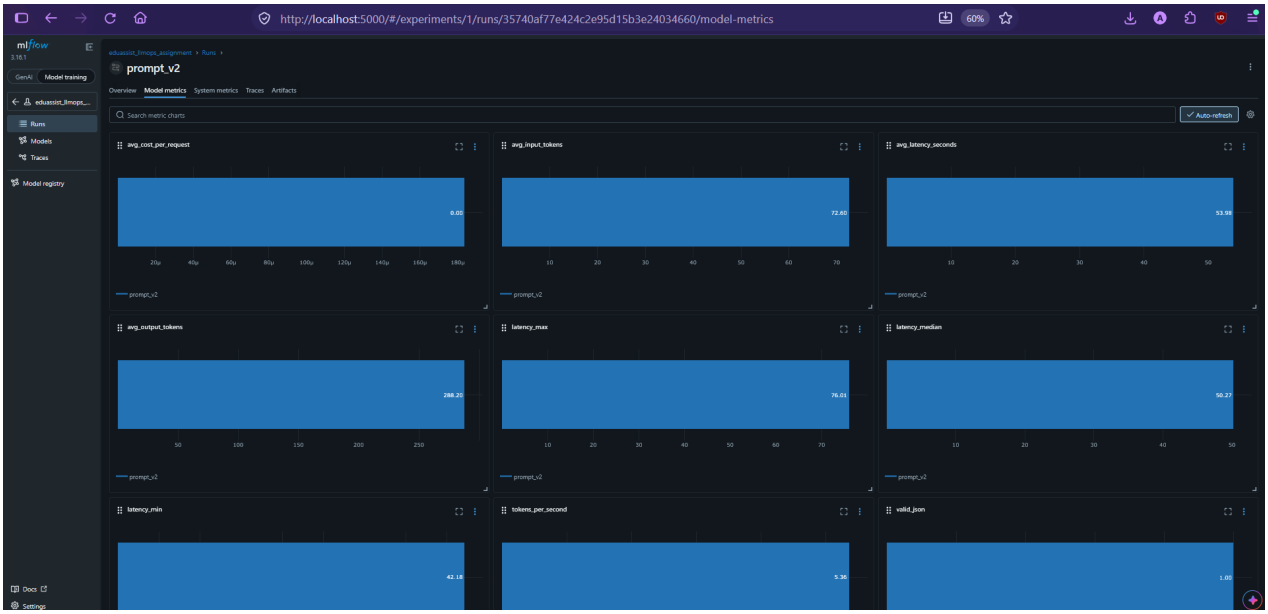


Figure 25 — MLflow prompt_v2: Model metrics chart (avg_latency_seconds, avg_input_tokens, etc.).

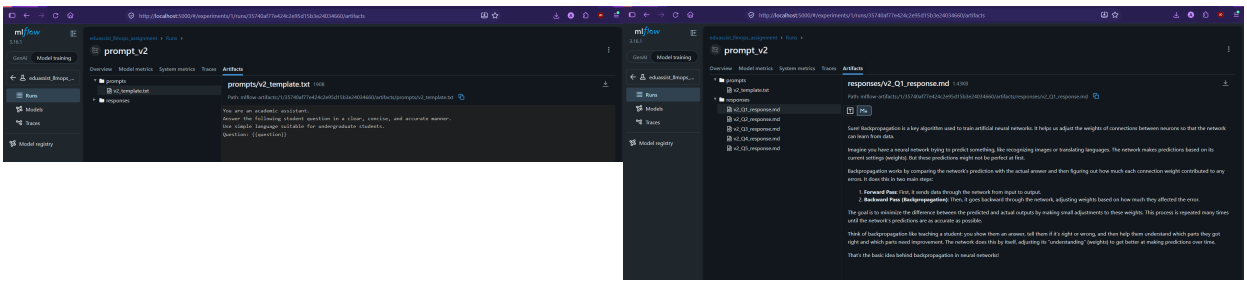


Figure 26 (left) — Artifacts › prompts/v2_template.txt (the registered prompt template).
Figure 27 (right) — Artifacts › responses (the five logged model responses).

9.3 — prompt_v3 run (production)

The prompt_v3 run overview lists the nine logged metrics; the metrics tab charts them over time; the artifacts tab stores the registered prompt template and the five logged model responses. v3 is the production-promoted version: it is the only one that meets the JSON-validity gate while minimising cost.

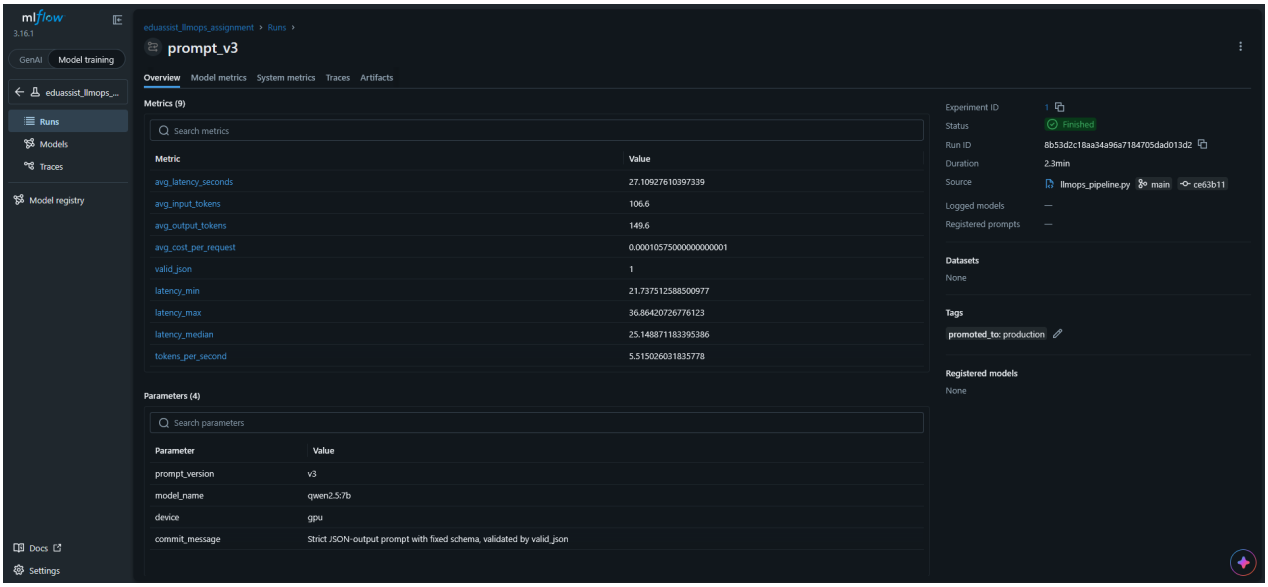


Figure 28 — MLflow prompt_v3 run: Overview tab with the nine logged metrics.

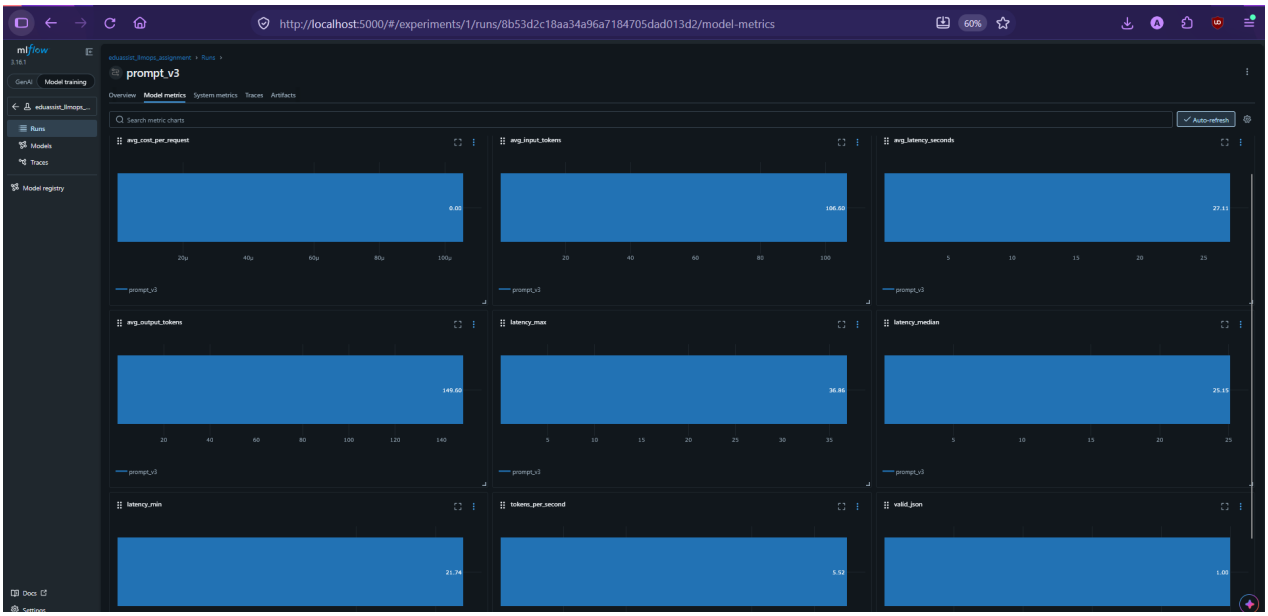


Figure 29 — MLflow prompt_v3: Model metrics chart (avg_latency_seconds, avg_input_tokens, etc.).

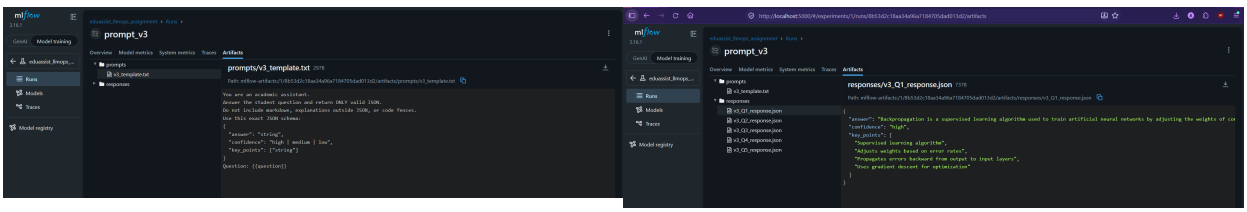


Figure 30 (left) — Artifacts › prompts/v3_template.txt (the registered prompt template).
Figure 31 (right) — Artifacts › responses (the five logged model responses).

10. Grafana Monitoring

The EduAssist LLMOps - Monitoring dashboard is auto-provisioned into Grafana and has seven panels: Requests/min, p95 Latency, Throughput (tok/s), JSON Validity, Latency distribution, Tokens (in/out) per request and Cost per request. All PromQL queries use the eduassist_* metric namespace defined in src/metrics.py (PROM_NAMESPACE = "eduassist"), so the dashboard and the exposed metrics match by construction. Background traffic from scripts/generate_traffic.py populates the panels. The low Requests/min (~4 req/min) visible below reflects the deployment fault in Section 12: most /ask calls returned HTTP 502 because the API container could not reach Ollama, so the success-path histograms stayed empty.

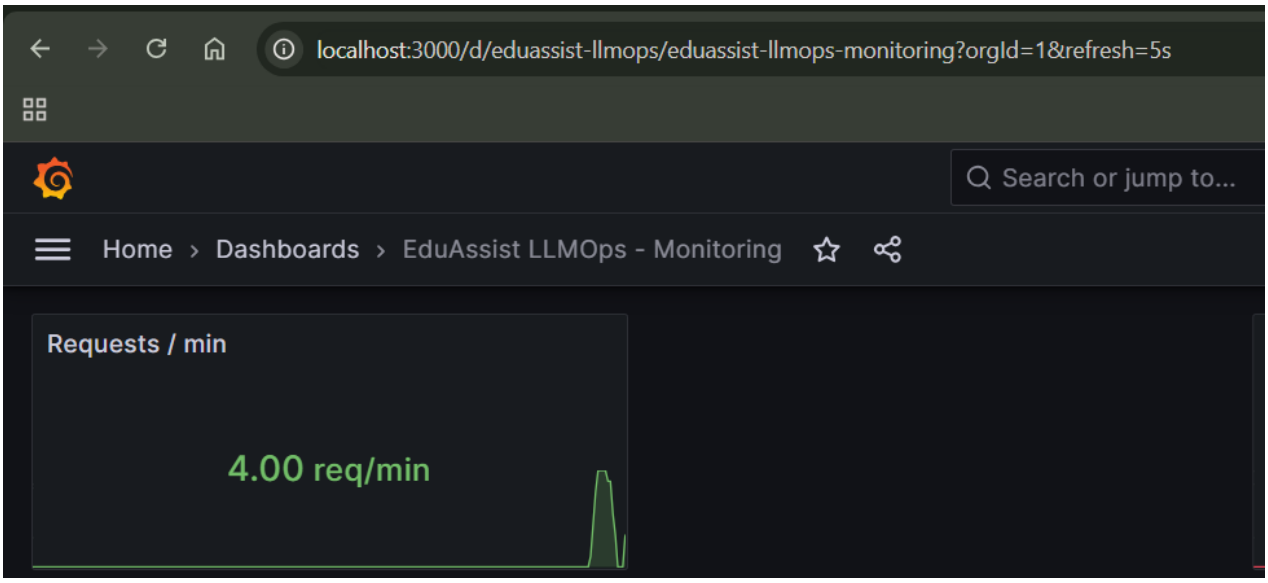


Figure 32 — Grafana: the EduAssist LLMOps Monitoring dashboard. The low Requests/min reflects the Section 12 fault (most /ask calls returned 502), which is resolved by the extra_hosts fix.

11. CI/CD

The .github/workflows/ci.yml workflow runs on every push and pull request: flake8 lint, pytest (25 unit tests), a Docker image build, and docker-compose config plus Prometheus rule validation. The mlflow==3.16.0 pin is enforced in requirements.txt and requirements-serve.txt; the Docker MLflow image is ghcr.io/mlflow/mlflow:v2.16.1 in docker-compose.yml. This mirrors the Assignment-1 CI pattern adapted for an LLM workload.

12. Known Issues and Precise Fixes (Deployment Error)

When ./initial_deploy.sh reaches Stage 9 on the Prayogshala Linux host, three errors appear in sequence: deployment failed near line 238 (exit 1), near line 240 (exit 1), and a curl 502 ("Expecting value: line 1 column 1") at line 241. The root cause is that the API container cannot reach the host Ollama: docker-compose.yml line 30 sets OLLAMA_HOST=http://host.docker.internal:11434, but Linux Docker Engine does not resolve host.docker.internal by default. With no route to Ollama, every /ask call raises HTTPException(502) at src/app.py line 88, the success-path metrics are never observed, and the Grafana panels depending on them stay empty. The table below gives the exact files and line numbers to change.

#	File	Line(s)	Problem	Fix
1 (root)	docker-compose.yml	24–39 (api service)	No extra_hosts; host.docker.internal (line 30) unresolved on Linux	Add extra_hosts: ["host.docker.internal:host-gateway"] to the api service
2	src/app.py	49–59	startup calls warmup_model() synchronously; timeout=600 blocks the event loop, /health never serves within 120 s	Run warm-up in a background thread; lower warm-up timeout to ~30 s
3	src/app.py	62–64	/health returns 200 even when Ollama is unreachable, masking the real failure	Probe Ollama (GET /api/tags); return 503 when unreachable so Stage 9 fails fast at line 238
4	src/app.py	88	raise HTTPException(502) is the 502 that smoke_test.sh:10-11 fails to parse as JSON	Upstream fix #1 removes the 502; optionally make smoke_test.sh lines 10-11 tolerate non-200
5	src/ollama_client.py	109, 139	timeout=600 for warm-up and /ask; if host resolves but Ollama is down, a 600 s hang blocks	Lower default timeout to ~30 s

Table 3. Precise file:line fixes for the Stage-9 deployment errors.

> **Minimum fix: change #1 alone (add extra_hosts to the api service in docker-compose.yml) makes Stage 9 pass and populates the Grafana panels.**

Prometheus note: monitoring/prometheus.yml lines 9–14 scrape api:8000/metrics via Docker DNS — this is correct and reachable. The "API does not seem reachable for Prometheus" symptom is actually Prometheus scraping the API but recording only the error counter (eduassist_requests_total{status="error"}) because the success-path histograms at src/app.py lines 90–100 are never observed when /ask returns 502. Grafana note: the eduassist_dashboard.json panels query eduassist_* metrics that match src/metrics.py lines 68–96 (PROM_NAMESPACE="eduassist") — there is no name mismatch; the empty panels are a consequence of fix #1, not a dashboard bug.

Repository: github.com/anmol-sinha-coder/BITS-MLOps-AIMLZG523_Assignment-2 · Executed on the BITS Prayogshala Virtual Lab · Anmol Sinha (2025AE05129) · AIMLZG523 (MLOps) · November 2026